On Efficient Version Control for Geometric Data: Literature Survey

BO HUANG, University of California, Riverside, Group: hhh, Student ID: 862469107

Due to the development of geo-positioning technologies, the spatial data such as points, trajectories and polygons become more and more important in our daily life. They are essential components of many high-level applications. For example, they can be used for recommending point-of-interests (POIs), designing travel routes, and analysing travel patterns. However, many of the existing studies focus on dealing with the static data: which means the data must be stored before performing any computation on it, and this is far from the reality situations. The location of points, the shape of trajectories and polygons may change as time goes by.

To manage the changes of spatial data efficiently, persistent (or functional) data structures may be a good choice. These kinds of data structures can store every change of the data to a new version and answer the changes between different versions efficiently. However, there are a few of studies tried to apply persistent data structures on geometric data. Therefore, I want to study the possibility of applying persistent (or functional) data structures for managing geometric data.

Additional Key Words and Phrases: Data Structures, Geometric Data Management, Version Control

ACM Reference Format:

Bo Huang. 2026. On Efficient Version Control for Geometric Data: Literature Survey. In . ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 RELATED WORK

This section discusses the related works about my topic. My topic is related to efficient management of geometric data. There are several efforts made in this direction, but they are lack of efficient version control and dynamic maintenance. Some studies focus on reducing the query processing dimension, but lead to more query time and hardness for version control. Last but not least, there are some data structures proposed for efficient version control, but they have limitations when be applied to geometric data. The related studies could be divided into three categories:

Method	Category	Work Dimension	Version Control	Theoretical Guarantee
Quad-trees [5]	Spatial Tree	Multi-dimensional	No Support	No Concrete Guarantee
k-d trees [1]	Spatial Tree	Multi-dimensional	No Support	Concrete Guarantee
Z (Morton)-Order [10, 11]	Space Filling Curve	Multi-dimensional	No Support	Depends on Applications
Hilbert-Order [8]	Space Filling Curve	Multi-dimensional	No Support	Depends on Applications
Persistent Search Tree [4]	Persistent Data Structure	One-dimensional	Support	Concrete Guarantee
Persistent Red-black Tree [12]	Persistent Data Structure	Two-dimensional	Support	Concrete Guarantee

Table 1. Brief Comparisons of Related Studies

(1) Spatial Tree: The category contains several search trees (both balanced and unbalanced) for managing spatial data. The key idea of this kind of method is that prune the space or points which are impossible to be the query result based on the application. The main drawbacks of them is that most of them are complex in both implementation and theoretical analysis. Moreover, adding dynamic maintenance and version control to them is quite difficult due to the complexity of higher dimension.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Association for Computing Machinery.

Manuscript submitted to ACM

(2) Space Filling Curve: Different from the previous ones, the space filling curve (SFC in short) aims to reduce the dimension of query processing. The key idea is using a curve in 1-dimensional space to pass all points in multi-dimensional space exactly once. After that, we can find a mapping from multi-dimensional space to 1-dimensional space. Although some extra effort need to be paid for query processing, the space filling curves can significantly reduce the difficulties of managing objects in multi-dimensional space.

(3) Persistent Data Structure: The methods in this category mainly focus on dealing with version control of data. When new version is generated, instead of storing an entire new data, only the changed data will be save incrementally to the older version. Therefore the persistent data structures are space efficient. But their disadvantages are the complexity in dealing with the tree structures (e.g., node rotation, node reconstruction, etc.). Therefore, most of them are mainly focus on the 1-dimensional data.

A brief comparison of the related studies are shown in Table 1. This section is organized as follows: The first category will be discussed in Section 1.1. The related studies about the second category are mentioned in Section 1.2. Finally, several methods of the third category will be presented in Section 1.3.

1.1 Spatial Tree

Just like the well-known binary search trees (BST) such as red-black trees [2] for managing one-dimensional data, there are also several multi-dimensional trees proposed for managing multi-dimensional data. Quad-trees [5] are a kind of space-partitioning search tree structure. Each tree node corresponds to a small region (or area) of the space. Each internal tree node (except the leaf nodes) has exactly four children. Since it is proposed to answer some map-based queries, the four children are usually represent the North East (NE), North West (NW), South East (SE), and South West (SW) sub regions. When we search along the quad-tree, we can skip an entire subtree if we found our query can not in its corresponding region. However, since quad-tree is based on space partition, it can not provide well theoretical guarantee on the search. To provide better theoretical guarantees, k-d trees (short for k-dimensional tree) are proposed [1]. Instead of space partition, k-d tree use point-based (or record-based) partition. Each node of k-d tree contains to a single point and a corresponding bounding region. And each internal node has exactly two children. The points covered by the bounding region is separated into two groups based on some separation strategy (usually is the coordinate median at some dimension of the point), and each child covers one group. Unlike quad-trees, k-d trees have well theoretical guarantees for the query processing. For managing more complex geometric objects such as polygons, R-tree are proposed [7]. R-trees allow more flexible space partition strategies, and thus they have many variants for handling different data. However, the existing spatial trees are either hard to be maintained in supporting dynamic insertions and deletions (quad-trees and k-d trees) or need large amount of resources (R-trees).

1.2 Space Filling Curve

Instead of dealing with the geometric data in their defined (original) space, some people made another approach. Given a multi-dimensional space (for example, Euclidean space), a space filling curve is a mapping which can map the multi-dimensional space to one-dimensional space (i.e., a line or curve). The space filling curve will start from some point in the given multi-dimensional space and passes all points in that space exactly once. Figure 1 shows several examples of space filling curves for two dimensional space. As you can see from the examples, you can either compute the space filling curves either sequentially (example (a) and (b)) or recursively (example (c), (d), and (e)). The space filling curves are first proposed by Peano [11]. The original one is mapping a unit interval (one-dimensional) to a square

(two-dimensional), which is the opposite direction and with limited mapping range as the space filling curves we usually discuss today. Orenstein later extend the mapping range with the similar order [10]. Hilbert [8] generalize the mapping to the entire space and as shown in Figure 1(e), the Hilbert space filling curve can be compute recursively. Recently, a composed method of combining space fill curve and k-d tree is proposed [3]. The key idea is that using a tree structure to manage several sub regions. Inside each region, they use the Peano method (also called Z-Curve, Morton-Order) to maintain the data efficiently in parallel. This method combined the benefits of both spatial trees and space filling curves. Similar idea are used in [6] which use space filling curve as heuristics for R-tree [7] construction. With the development of geo-spatial technologies, more and more research studies and space filling curves are proposed for managing geometric objects efficiently . Since the space filling curves can help us reduce the problem dimension and make multi-dimensional data easier for handling. After getting the one-dimensional data, we can easily handle them using existing binary search trees such as red-black trees [2].

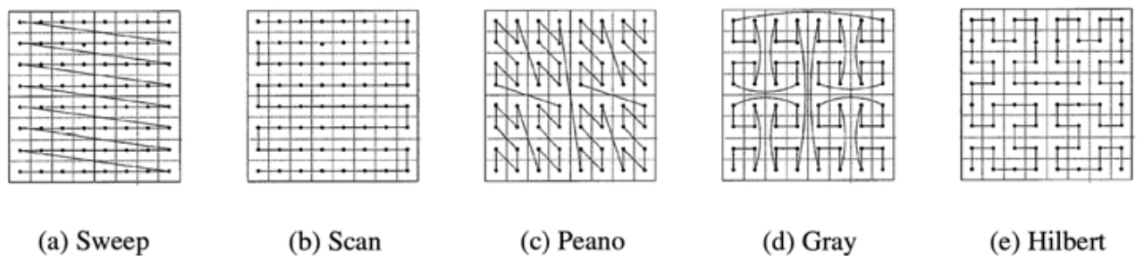

Fig. 1. Examples of Space Filling Curves [9]

1.3 Persistent data structure

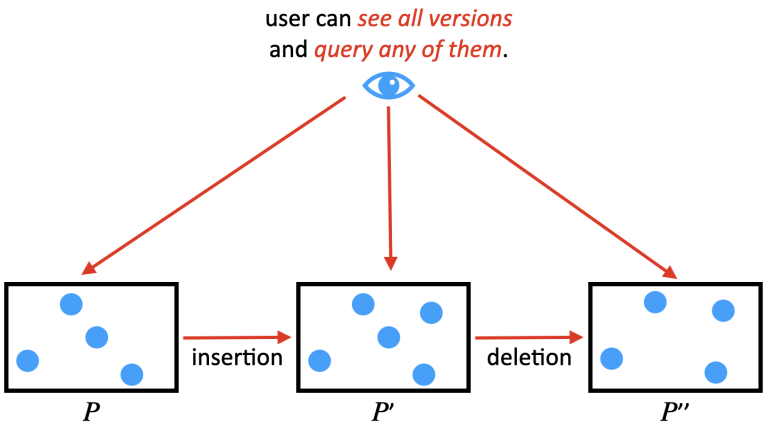

Fig. 2. Examples of Saving All Versions

A persistent data structure is a data structure which keeps are previous versions of itself when modification happens. As Figure 2 shows, after several modifications on the data structure, the user can still query the older version. This is very

useful in real-world applications. For example, user may need to make a query that "how many buildings in the given region at 2004". In this case, we can not directly use the latest version of data (in 2023) to answer it. A straightforward approach is that we can store all versions simultaneously, however this will lead to large space consumption (since we store many duplicate data). The first persistent binary search tree is proposed in [4]. The key idea is that for each version of the data structure, the tree will make a new branch at the modified node. Therefore, the new version and older version can share some "static" data, instead of store every again (this method called path copying). Persistent data structure can also be used for processing geometric data. In [12], the Sarnak and Tarjan proposed a method of using persistent red-black trees for point location query in 2-dimensional space. However, their method cannot deal with more general location-based queries.

REFERENCES

[1] Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Commun. ACM* 18, 9 (1975), 509–517.

[2] PE Black. 2006. Red Black Tree.

[3] Guy E. Blelloch and Magdalen Dobson. 2022. Parallel Nearest Neighbors in Low Dimensions with Batch Updates. In *Proceedings of the Symposium on Algorithm Engineering and Experiments, ALENEX 2022, Alexandria, VA, USA, January 9-10, 2022*, Cynthia A. Phillips and Bettina Speckmann (Eds.). SIAM, 195–208. https://doi.org/10.1137/1.9781611977042.16

[4] James R Driscoll, Neil Sarnak, Daniel Dominic Sleator, and Robert Endre Tarjan. 1986. Making data structures persistent. In *Proceedings of the eighteenth annual ACM symposium on Theory of computing*. 109–121.

[5] Raphael A Finkel and Jon Louis Bentley. 1974. Quad trees a data structure for retrieval on composite keys. *Acta informatica* 4 (1974), 1–9.

[6] Yan Gu, Yong He, Kayvon Fatahalian, and Guy E. Blelloch. 2013. Efficient BVH Construction via Approximate Agglomerative Clustering. In *High-Performance Graphics 2013, Anaheim, California, USA, July 19-21, 2013. Proceedings*, Kayvon Fatahalian, Christian Theobalt, and Jaakko Lehtinen (Eds.). ACM, 81–88. https://doi.org/10.1145/2492045.2492054

[7] Antonin Guttman. 1984. R-trees: A dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD international conference on Management of data*. 47–57.

[8] David Hilbert and David Hilbert. 1935. Über die stetige Abbildung einer Linie auf ein Flächenstück. *Dritter Band: Analysis· Grundlagen der Mathematik· Physik Verschiedenes: Nebst Einer Lebensgeschichte* (1935), 1–2.

[9] Mohamed F. Mokbel, Walid G. Aref, and Ibrahim Kamel. 2003. Analysis of Multi-Dimensional Space-Filling Curves. *GeoInformatica* 7, 3 (2003), 179–209. https://doi.org/10.1023/A:1025196714293

[10] Jack A Orenstein and Tim H Merrett. 1984. A class of data structures for associative searching. In *Proceedings of the 3rd ACM SIGACT-SIGMOD Symposium on Principles of Database Systems*. 181–190.

[11] Giuseppe Peano and G Peano. 1990. *Sur une courbe, qui remplit toute une aire plane*. Springer.

[12] Neil Sarnak and Robert E Tarjan. 1986. Planar point location using persistent search trees. *Commun. ACM* 29, 7 (1986), 669–679.